

LAPORAN TUGAS BESAR

Implementasi dan Analisis Kinerja Sistem Client–Proxy–Server Berbasis Socket Python:
Evaluasi Protokol TCP/UDP dan Parameter Quality of Service



Mata Kuliah: Jaringan Komputer

Kelas: IT-48-05

Anggota:

1. Jonatannael Panjaitan (103032430036)
2. Sheren Aulia Azahra (103032400036)
3. Ririn Nur Aini (103032400054)

Program Studi S1 Teknologi Informasi

Fakultas Informatika

Telkom University

2026

DAFTAR ISI

BAB 1 PENDAHULUAN	1
1.1 Identitas Kelompok.....	1
1.2 Pembagian Tugas	1
BAB 2 Implementasi Sistem.....	2
2.1 Arsitektur Sistem	2
2.1.1 Gambaran Umum	2
2.1.2 Topologi Komunikasi.....	2
2.1.3 Konfigurasi Port	2
2.2 Web Server (webserver.py).....	3
2.2.1 TCP Server – HTTP	3
2.2.2 UDP Server – QoS Echo.....	4
2.2.3 Mekanisme Konkurensi.....	4
2.3 Proxy Server (proxy.py).....	4
2.3.1 Mekanisme Forwarding.....	5
2.3.2 Mekanisme Caching	6
2.3.3 Penanganan Galat.....	8
2.4 Client (client.py)	10
2.4.1 Mode HTTP (TCP).....	10
2.5 Justifikasi Desain	10
2.5.1 Penanganan Konkurensi.....	10
2.5.2 Strategi Error Handling	11
BAB 3 Analisis QoS dan Multithreading.....	13
3.1 Hasil Pengukuran QoS	13
3.1.1 Tabel Data QoS	13
3.1.2 Analisis Hasil QoS.....	14
3.2 Analisis Cache HIT vs MISS	15
3.2 Skenario Single Client vs Multi-Client.....	16

3.2.1 Bukti Log Aktivitas Terminal	16
3.2.2 Analisis Perilaku dan Waktu Respons	16
3.2.3 Dampak Arsitektur Thread-per-Connection.....	17
3.4 Evaluasi Multithreading	17
3.4.1 Bukti Perilaku Thread.....	17
3.4.2 Konsistensi Cache pada Akses Konkuren	18
3.4.3 Skalabilitas dan Resource Usage	19
3.4.4 Analisis WiFi vs LAN	19
BAB 4 Troubleshooting dan Kendala Implementasi.....	20
BAB 5 Kesimpulan dan Saran.....	22
5.1 Kesimpulan	22
5.2 Saran	23
DAFTAR PUSTAKA.....	24

BAB 1 PENDAHULUAN

1.1 Identitas Kelompok

No	Nama	NIM
1.	Jonatannael Panjaitan	103032430036
2.	Sheren Aulia Azahra	103032400036
3.	Ririn Nur Aini	103032400054

1.2 Pembagian Tugas

Tabel berikut menjelaskan kontribusi masing-masing anggota dalam proses implementasi dan pengujian sistem.

No	Nama	Peran	Kontribusi
1.	Jonatannael Panjaitan	Webserver.py	Implementasi TCP HTTP server, UDP echo server, dan mekanisme konkurensi (thread-per-connection).
2.	Sheren Aulia Azahra	Client.py	Implementasi client mode HTTP (TCP) dan mode QoS (UDP piger) berserta perhitungan parameter QoS.
3.	Ririn Nur Aini	Proxyserver.py + Wireshark	Implementasi proxy forwarding & caching, serta dokumentasi pengujian (capture & analisis) menggunakan Wireshark.

BAB 2 Implementasi Sistem

2.1 Arsitektur Sistem

2.1.1 Gambaran Umum

Sistem diimplementasikan menggunakan arsitektur Client–Proxy–Server di mana seluruh komunikasi antara client dan web server wajib melewati proxy sebagai perantara terpusat.

- Client (client.py): menginisiasi HTTP request ke Proxy dan menjalankan modul UDP pinger untuk QoS.
- Proxy Server (proxy.py): bertindak sebagai gateway, mengelola caching, dan meneruskan request ke Web Server.
- Web Server (webserver.py): melayani file statis melalui TCP dan menyediakan layanan UDP echo server.

2.1.2 Topologi Komunikasi

Alur komunikasi pada sistem dapat digambarkan sebagai berikut:

CLIENT —HTTP/TCP—► PROXY (8081) —HTTP/TCP—► WEB SERVER (8001)
CLIENT —UDP Ping—► WEB SERVER (9002, echo)

Permintaan HTTP selalu menempuh jalur Client → Proxy → Web Server, sedangkan lalu lintas UDP untuk pengukuran QoS dikirim langsung dari client ke web server.c

2.1.3 Konfigurasi Port

Komponen	Port	Protokol	Keterangan
Web Server	8001	TCP	HTTP server
Web Server	9002	UDP	QoS Echo server
Proxy Server	8081	TCP	Forward + Cache
Client	Ephemeral	TCP + UDP	HTTP + UDP Pinger

2.2 Web Server (webserver.py)

2.2.1 TCP Server – HTTP

Web server membuka socket TCP (SOCK_STREAM) dan melakukan bind ke 0.0.0.0:8001 dengan opsi SO_REUSEADDR agar port dapat segera digunakan kembali setelah server dihentikan. Setelah menerima koneksi, server membaca request, mem-parsing baris pertama HTTP (mis. “GET /index.html HTTP/1.1”) untuk memperoleh method dan path, kemudian memetakan path menjadi nama berkas pada direktori kerja.

Jika path kosong (akses ke “/”), server mengembalikan berkas default index.html. Apabila berkas tersedia, server membaca berkas dalam mode biner (rb) dan menyusun respons HTTP 200 OK lengkap dengan header Content-Type, Content-Length, dan body berkas. Apabila berkas tidak ditemukan, server mengembalikan respons 404 Not Found; bila terjadi kesalahan internal atau request malformed, server mengembalikan 500 Internal Server Error.

Penentuan Content-Type (MIME type). Server tidak menggunakan Content-Type yang sama untuk semua berkas, melainkan memetakan ekstensi berkas ke MIME type yang sesuai (mis. .html → text/html, .css → text/css, .js → application/javascript, .png → image/png). Pemetaan ini penting agar browser dapat menerapkan stylesheet dan skrip dengan benar (lihat Bab 4, Troubleshooting).

```
MIME_TYPES = {  
    '.html': 'text/html; charset=utf-8',  
    '.htm': 'text/html; charset=utf-8',  
    '.css': 'text/css; charset=utf-8',  
    '.js': 'application/javascript; charset=utf-8',  
    '.json': 'application/json; charset=utf-8',  
    '.png': 'image/png',  
    '.jpg': 'image/jpeg',  
    '.jpeg': 'image/jpeg',  
    '.gif': 'image/gif',  
    '.svg': 'image/svg+xml',  
    '.ico': 'image/x-icon',  
    '.mp4': 'video/mp4',  
    '.webm': 'video/webm',  
    '.woff': 'font/woff',  
    '.woff2': 'font/woff2',  
    '.txt': 'text/plain; charset=utf-8',  
}
```

2.2.2 UDP Server – QoS Echo

Layanan UDP echo membuka socket UDP (SOCK_DGRAM) pada port 9002. Server menunggu datagram dari client menggunakan `recvfrom()`, lalu memantulkan (echo) pesan yang sama persis kembali ke alamat pengirim menggunakan `sendto()`. Karena UDP bersifat connectionless dan tidak menjamin pengiriman, layanan echo ini menjadi dasar pengukuran RTT, jitter, throughput, dan packet loss pada sisi client.

2.2.3 Mekanisme Konkurensi

Server menjalankan layanan TCP dan UDP pada thread terpisah. Untuk layanan TCP, setiap koneksi client yang diterima ditangani oleh thread baru (model thread-per-connection) yang dibuat sebagai daemon thread, sehingga beberapa client dapat dilayani secara bersamaan tanpa saling memblokir. Setiap socket koneksi diberi timeout (`settimeout`) untuk mencegah thread menggantung pada koneksi yang tidak responsif. Server mengadopsi model *thread-per-connection* menggunakan modul `threading`. Pada `start_tcp_server()`, setiap koneksi masuk yang diterima melalui `accept()` akan didelegasikan ke *thread* baru (`handle_tcp_client`)

```
client_thread = threading.Thread(target=handle_tcp_client, args=(...));  
client_thread.daemon = True
```

Dimana mekanisme konkurensi memungkinkan server melayani beberapa client secara simultan

2.3 Proxy Server (proxy.py)

Proxy server merupakan komponen perantara antara client dan web server. Pada sistem ini, proxy server berjalan pada alamat 0.0.0.0 dengan port 8081, sehingga dapat menerima koneksi dari client yang berada dalam jaringan yang sama. Setelah menerima request dari client, proxy akan meneruskan request tersebut ke web server yang berjalan pada alamat 10.169.91.1 dengan port 8001.

Selain berfungsi sebagai penerus request, proxy server juga memiliki mekanisme caching berbasis berkas. Dengan adanya caching, respons dari web server dapat disimpan secara lokal oleh proxy. Apabila client meminta resource yang sama pada waktu berikutnya, proxy dapat langsung mengirimkan respons dari cache tanpa perlu menghubungi web server kembali. Mekanisme ini bertujuan untuk mengurangi beban web server dan mempercepat waktu respons pada request berikutnya.

Proxy server juga dilengkapi dengan penanganan galat, seperti kondisi ketika web server tidak dapat dijangkau atau ketika koneksi ke web server mengalami timeout. Dengan demikian, client tetap memperoleh respons HTTP yang sesuai meskipun terjadi gangguan pada sisi web server.

2.3.1 Mekanisme Forwarding

Proxy menerima request HTTP dari client pada port 8081, lalu membuka koneksi TCP baru ke web server (8001) untuk meneruskan request tersebut. Respons dari web server diteruskan kembali ke client tanpa mengubah isi (body).

Mekanisme forwarding dimulai ketika proxy menerima koneksi dari client melalui port 8081. Setiap koneksi client ditangani menggunakan thread terpisah melalui fungsi `handle_client()`. Dengan pendekatan ini, proxy dapat melayani lebih dari satu client secara bersamaan tanpa harus menunggu satu request selesai terlebih dahulu.

Request dari client diterima menggunakan perintah:

```
request_data = client_socket.recv(BUFFER_SIZE)
```

Data request tersebut kemudian diubah menjadi string dengan decoding UTF-8 menggunakan parameter `errors='ignore'`. Hal ini dilakukan agar proxy tetap dapat memproses request meskipun terdapat karakter tertentu yang tidak dapat didekode secara sempurna.

```
request_str = request_data.decode('utf-8', errors='ignore')
first_line = request_str.split('\n')[0]
url = first_line.split(' ')[1]
```

Proxy mengambil baris pertama dari HTTP request, misalnya:

```
GET /index.html HTTP/1.1
```

Dari baris tersebut, proxy mengambil bagian path atau URL, yaitu `/index.html`. Path inilah yang digunakan untuk menentukan resource yang diminta client sekaligus menjadi dasar penamaan file cache.

Apabila client mengakses root URL `/`, maka proxy mengubahnya menjadi `index.html`. Hal ini disesuaikan dengan file default pada web server.

```
filename = url.lstrip('/')
if filename == "":
    filename = 'index.html'
```

Jika resource belum tersedia di cache, proxy akan membuka koneksi TCP baru ke web server menggunakan socket. Koneksi tersebut diarahkan ke alamat dan port web server yang telah ditentukan pada konfigurasi:

```
WEB_SERVER_HOST = '10.169.91.1'
WEB_SERVER_PORT = 8001
```

Koneksi ke web server dibuat dengan perintah:


```

web_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
web_socket.settimeout(WEB_SERVER_TIMEOUT)
web_socket.connect((WEB_SERVER_HOST, WEB_SERVER_PORT))

```

Setelah koneksi berhasil dibuat, proxy meneruskan request asli dari client ke web server menggunakan:

```

web_socket.sendall(request_data)

```

Pada bagian ini, header HTTP dari client diteruskan apa adanya ke web server. Proxy tidak mengubah isi request header maupun body request. Hal ini penting karena informasi seperti metode HTTP, path, versi HTTP, dan header lain tetap diterima oleh web server sesuai request awal dari client.

Respons dari web server kemudian diterima oleh proxy secara bertahap menggunakan loop `recv()` sampai tidak ada data lagi yang dikirimkan oleh web server.

```

response_data = b""
while True:
    chunk = web_socket.recv(BUFFER_SIZE)
    if len(chunk) == 0:
        break
    response_data += chunk

```

Respons tersebut kemudian diteruskan kembali ke client menggunakan:

```

client_socket.sendall(response_data)

```

Proxy tidak mengubah isi respons dari web server. Artinya, header seperti Content-Type, Content-Length, status code, dan body tetap dikirimkan dalam bentuk asli. Hal ini membuat format file yang dikirimkan, seperti HTML, CSS, JavaScript, atau gambar, tetap dapat dikenali dengan benar oleh browser. Dengan demikian, risiko kerusakan Content-Type dapat dihindari karena proxy meneruskan respons web server secara utuh.

2.3.2 Mekanisme Caching

Proxy menyimpan salinan respons untuk request yang sama sehingga permintaan berikutnya dapat dilayani langsung dari cache (cache HIT) tanpa membebani web server (cache MISS).

Proxy server menggunakan mekanisme caching berbasis berkas atau file-based caching. Setiap respons dari web server akan disimpan sebagai file cache di penyimpanan lokal proxy. Cache ini digunakan untuk melayani request berikutnya apabila client meminta resource yang sama.

Kunci cache ditentukan berdasarkan path URL yang diminta oleh client. Path tersebut diubah menjadi nama file cache dengan menambahkan prefix `cache_` dan mengganti karakter `/` menjadi `_`.

```
cache_filename = "cache_" + filename.replace('/', '_')
```

Sebagai contoh, apabila client meminta:

```
/index.html
```

maka nama file cache yang dibuat adalah:

```
cache_index.html
```

Apabila client meminta file pada subdirektori, misalnya:

```
/assets/style.css
```

maka nama file cache menjadi:

```
cache_assets_style.css
```

Sebelum meneruskan request ke web server, proxy akan memeriksa apakah file cache sudah tersedia menggunakan:

```
if os.path.exists(cache_filename):
```

Jika file cache ditemukan, maka kondisi tersebut disebut sebagai cache HIT. Pada kondisi cache HIT, proxy tidak menghubungi web server. Proxy langsung membaca isi file cache dan mengirimkannya ke client.

```
with open(cache_filename, 'rb') as f:  
    response_data = f.read()  
    client_socket.sendall(response_data)
```

Karena file cache menyimpan respons secara utuh dalam bentuk byte, maka header HTTP dan body respons tetap tersimpan. Hal ini membuat informasi penting seperti Content-Type tetap tersedia saat respons dikirim dari cache ke client.

Sebaliknya, jika file cache belum tersedia, maka kondisi tersebut disebut sebagai cache MISS. Pada kondisi cache MISS, proxy akan membuka koneksi ke web server, meneruskan request client, menerima respons dari web server, kemudian menyimpan respons tersebut ke dalam file cache.

```
with open(cache_filename, 'wb') as f:  
    f.write(response_data)
```

Setelah disimpan, respons tetap dikirimkan ke client. Dengan demikian, request pertama terhadap suatu resource akan menjadi cache MISS, sedangkan request berikutnya untuk resource yang sama akan menjadi cache HIT.

Pada implementasi ini, cache tidak memiliki mekanisme expiry atau masa kedaluwarsa otomatis. Artinya, file cache akan tetap digunakan selama file tersebut masih ada di penyimpanan lokal proxy. Jika isi file pada web server berubah, proxy tetap dapat mengirimkan versi lama dari cache sampai file cache tersebut dihapus secara manual atau mekanisme invalidasi tambahan diterapkan.

Untuk menjaga konsistensi saat terdapat beberapa client yang mengakses resource yang sama secara bersamaan, proxy menggunakan `threading.Lock()`.

```
cache_lock = threading.Lock()
```

Lock digunakan ketika proxy menulis file cache:

```
with cache_lock:
    if not os.path.exists(cache_filename):
        with open(cache_filename, 'wb') as f:
            f.write(response_data)
```

Penggunaan lock ini bertujuan untuk mencegah race condition, yaitu kondisi ketika dua atau lebih thread mencoba menulis file cache yang sama pada waktu bersamaan. Selain itu, terdapat pengecekan ulang setelah lock berhasil diperoleh. Pengecekan ini memastikan bahwa file cache belum dibuat oleh thread lain sebelum proses penulisan dilakukan.

Dengan mekanisme tersebut, caching pada proxy menjadi lebih aman untuk akses konkuren dan dapat mengurangi kemungkinan file cache rusak akibat proses tulis bersamaan.

2.3.3 Penanganan Galat

Proxy server dilengkapi dengan penanganan galat untuk mengantisipasi kondisi ketika web server tidak dapat dihubungi atau tidak memberikan respons dalam batas waktu tertentu. Penanganan galat ini penting agar client tetap menerima respons HTTP yang jelas, bukan hanya koneksi yang terputus tanpa informasi.

Pada saat proxy membuka koneksi ke web server, timeout telah ditentukan melalui konfigurasi:

```
WEB_SERVER_TIMEOUT = 10
```

Nilai tersebut berarti proxy akan menunggu respons dari web server maksimal selama 10 detik. Jika web server tidak merespons dalam batas waktu tersebut, maka proxy menangani kondisi ini sebagai timeout dan mengirimkan respons HTTP 504 Gateway Timeout kepada client.

```
except socket.timeout:  
    error_msg = "HTTP/1.1 504 Gateway Timeout ..."  
    client_socket.sendall(error_msg.encode('utf-8'))
```

Status 504 Gateway Timeout menunjukkan bahwa proxy berhasil menerima request dari client, tetapi tidak memperoleh respons dari web server dalam batas waktu yang ditentukan. Pada halaman error, client akan menerima informasi bahwa web server tidak merespons.

Selain timeout, proxy juga menangani kondisi ketika web server menolak koneksi atau sedang tidak aktif. Kondisi ini ditangani melalui `ConnectionRefusedError`.

```
except ConnectionRefusedError:  
    error_msg = "HTTP/1.1 502 Bad Gateway ..."  
    client_socket.sendall(error_msg.encode('utf-8'))
```

Status 502 Bad Gateway menunjukkan bahwa proxy tidak dapat meneruskan request karena web server tujuan tidak dapat dijangkau. Pada kondisi ini, client akan menerima halaman error yang menjelaskan bahwa web server sedang down.

Proxy juga memiliki penanganan galat umum melalui blok `except Exception as e`. Blok ini digunakan untuk menangani kesalahan lain yang mungkin terjadi selama proses penerimaan request, forwarding, caching, atau pengiriman respons.

```
except Exception as e:  
    error_msg = "HTTP/1.1 502 Bad Gateway ..."  
    client_socket.sendall(error_msg.encode('utf-8'))
```

Dengan adanya penanganan galat tersebut, proxy dapat memberikan respons yang lebih informatif kepada client. Selain itu, proxy juga mencatat pesan error pada terminal agar proses debugging lebih mudah dilakukan oleh pengembang.

Pada bagian akhir proses, proxy selalu menutup koneksi ke web server dan client melalui blok `finally`.

```
finally:  
    if web_socket:  
        web_socket.close()  
    client_socket.close()
```

Penutupan koneksi ini bertujuan untuk mencegah socket tetap terbuka setelah request selesai diproses. Dengan demikian, penggunaan resource jaringan menjadi lebih terkendali dan proxy dapat terus menerima koneksi baru dari client lain.

2.4 Client (client.py)

2.4.1 Mode HTTP (TCP)

Pada mode ini client membuat socket TCP, terhubung ke proxy (PROXY_HOST:8081), lalu menyusun dan mengirim request HTTP GET untuk berkas yang diminta pengguna dengan header Connection: close. Client kemudian membaca seluruh respons secara berulang menggunakan `recv()` hingga koneksi ditutup, dan menampilkan respons (header beserta body) ke layar.

```
request = f'GET {file_path} HTTP/1.1\r\nHost: {PROXY_HOST}\r\nConnection: close\r\n\r\n'
```

2.4.2 Mode QoS (UDP)

Pada mode QoS, client mengirim sejumlah paket “Ping” (default 10, minimum 10) ke UDP echo server (9002) dengan timeout 1 detik per paket. Untuk setiap paket, client mencatat waktu kirim dan waktu terima untuk menghitung Round Trip Time (RTT). Dari kumpulan RTT, client menghitung empat parameter QoS:

- **RTT:** nilai minimum, maksimum, dan rata-rata dari seluruh RTT paket yang berhasil.
- **Jitter:** standar deviasi dari selisih RTT antar-paket berurutan (variasi delay), sesuai definisi pada spesifikasi.
- **Throughput:** total byte diterima dikali 8, dibagi total duruui pengiriman (bit per detik, disajikan dalam kbps).
- **Packet Loss:** persentase paket yang tidak menerima balasan (timeout) terhadap total paket yang dikirim.

2.5 Justifikasi Desain

2.5.1 Penanganan Konkurensi

Model thread-per-connection dipilih karena sederhana, mudah dipahami, dan cukup untuk skala pengujian tugas besar. Penggunaan daemon thread memastikan seluruh thread otomatis berhenti ketika program utama dihentikan, sehingga tidak menyisakan thread yang menggantung (thread leak). Alternatif thread pool atau I/O asynchronous lebih hemat sumber daya pada beban tinggi, namun menambah kompleksitas yang tidak diperlukan pada skala ini.

Khusus pada komponen Proxy Server (proxy.py), model thread-per-connection ini memiliki urgensi operasional yang sangat krusial. Sebagai entitas perantara (middleman), proxy harus menangani dua tahapan I/O jaringan yang bersifat blocking secara simultan: menerima request dari client dan membuka koneksi baru (forwarding) ke Web Server. Jika proxy didesain menggunakan model single-threaded, satu client yang mengalami

latensi tinggi atau proses transfer file besar akan langsung memblokir seluruh antrean client lainnya (head-of-line blocking). Dengan mengalokasikan satu thread khusus untuk setiap client menggunakan threading.Thread, proxy dapat menyembunyikan latensi operasi I/O tersebut dan melayani banyak request client secara paralel tanpa saling menginterupsi.

2.5.2 Strategi Error Handling

Server dan proxy membedakan beberapa kelas respons galat untuk menjaga ketangguhan sistem dari kegagalan koneksi. Agar mekanisme deteksi dan dampak galat pada sistem Client–Proxy–Server ini mudah dipahami, berikut adalah pemetaan strategi penanganan galat yang diimplementasikan melalui blok *try-except-finally*:

Kode Status HTTP	Nama Galat (Mekanisme)	Kondisi Pemicu Jaringan / Socket	Strategi Respons & Dampak Sistem
404 Not Found	Server-Side Error	Berkas yang diminta oleh client tidak ditemukan pada direktori lokal Web Server.	Web Server menyajikan halaman 404.html secara konsisten ke client melalui proxy. Berkas ini tidak masuk ke dalam cache proxy karena statusnya bukan 200 OK.
500 Internal Server Error	Server-Side Exception	Terjadi malformed request (gagal parsing HTTP) atau exception tidak terduga pada internal Web Server.	Web Server menangkap gangguan melalui blok pengecualian umum dan menyajikan 500.html untuk mencegah crash pada socket utama.
502 Bad Gateway	Proxy-Side Connection Failure	Terjadinya ConnectionRefusedError atau exception umum saat proxy mencoba melakukan jabatan tangan TCP (connect()) ke IP Web Server yang sedang mati/down.	Proxy bertindak sebagai pelindung dengan langsung menyusun raw byte stream halaman HTML galat 502 secara mandiri, lalu mengirimkannya ke client agar koneksi client tidak menggantung.

504 Gateway Timeout	Proxy-Side Timeout	Web Server tidak memberikan respons data dalam rentang batas aman WEB_SERVER_TIMEOUT yang ditentukan (10 detik).	Pengecualian socket.timeout terpicu pada proxy. Thread proxy akan menghentikan proses blocking recv(), membuat respons HTML 504 darurat, dan mengirimkannya ke client sebelum menutup koneksi.
---------------------------	-----------------------	--	--

Melalui penerapan blok `finally` di setiap akhir pemrosesan *thread*, sistem menjamin bahwa `client_socket` maupun `web_socket` yang sempat terbuka akan selalu dieksekusi dengan perintah `close()`. Strategi ini memastikan pembebasan alokasi deskriptor berkas (*file descriptor*) pada sistem operasi perantara, sehingga mencegah terjadinya penumpukan sisa koneksi mati yang dapat menurunkan performa sistem secara keseluruhan.

BAB 3 Analisis QoS dan Multithreading

3.1 Hasil Pengukuran QoS

3.1.1 Tabel Data QoS

Pengukuran dilakukan dengan mengirim sejumlah paket UDP ke echo server pada jaringan LAN (Local Area Network) menggunakan media kabel untuk menjamin stabilitas throughput dan meminimalkan faktor interferensi luar.

Skenario	RTT Min (ms)	RTT Avg (ms)	RTT Max (ms)	Jitter / Loss / Throughput
LAN, 10 paket	4.75 ms	68.25 ms	245.14 ms	Jitter: 107.72 ms Throughput: 2.86 kbps Loss: 0.0%
LAN, 50 paket	5.73 ms	44.94 ms	422.97 ms	Jitter: 79.62 ms Throughput: 4.53 kbps Loss: 0.0%

```
=====
PROGRAM CLIENT MULTI-MODE
=====
Pilih mode operasi:
1. Mode HTTP (TCP) -> Akses Web via Proxy
2. Mode QoS (UDP) -> Uji Pinger ke Server
3. Keluar
Masukkan pilihan (1/2/3) : 2

--- Mode Analisis QoS (UDP) ---
Berapa paket yang ingin dikirim? (Tekan enter untuk default 10) : 10

[CLIENT] Memulai pengiriman 10 paket UDP ke 10.169.91.1:9002...
Reply dari 10.169.91.1: seq=1 time=51.99 ms payload="Ping 1 1781436865.673112"
Reply dari 10.169.91.1: seq=2 time=10.90 ms payload="Ping 2 1781436865.7254145"
Reply dari 10.169.91.1: seq=3 time=188.80 ms payload="Ping 3 1781436865.736464"
Reply dari 10.169.91.1: seq=4 time=245.14 ms payload="Ping 4 1781436865.925392"
Reply dari 10.169.91.1: seq=5 time=13.96 ms payload="Ping 5 1781436866.170679"
Reply dari 10.169.91.1: seq=6 time=80.61 ms payload="Ping 6 1781436866.1848948"
Reply dari 10.169.91.1: seq=7 time=5.66 ms payload="Ping 7 1781436866.265787"
Reply dari 10.169.91.1: seq=8 time=4.75 ms payload="Ping 8 1781436866.2715223"
Reply dari 10.169.91.1: seq=9 time=6.96 ms payload="Ping 9 1781436866.2764254"
Reply dari 10.169.91.1: seq=10 time=73.77 ms payload="Ping 10 1781436866.2863617"

=====
HASIL ANALISIS QoS
=====
RTT      : Min = 4.75 ms | Max = 245.14 ms | Rata-rata = 68.25 ms
Jitter   : 107.72 ms
Throughput : 2.86 kbps (246 bytes dalam 0.6873 detik)
Packet Loss: 0.0% (10/10 paket berhasil diterima)
=====
```

```
Reply dari 10.169.91.1: seq=45 time=10.78 ms payload="Ping 45 1781436881.4098885"
Reply dari 10.169.91.1: seq=46 time=6.02 ms payload="Ping 46 1781436881.4208608"
Reply dari 10.169.91.1: seq=47 time=6.17 ms payload="Ping 47 1781436881.4289412"
Reply dari 10.169.91.1: seq=48 time=6.57 ms payload="Ping 48 1781436881.435203"
Reply dari 10.169.91.1: seq=49 time=9.89 ms payload="Ping 49 1781436881.4437027"
Reply dari 10.169.91.1: seq=50 time=8.22 ms payload="Ping 50 1781436881.453765"

=====
HASIL ANALISIS QoS
=====
RTT      : Min = 5.73 ms | Max = 422.97 ms | Rata-rata = 44.94 ms
Jitter   : 79.62 ms
Throughput : 4.53 kbps (1282 bytes dalam 2.2660 detik)
Packet Loss: 0.0% (50/50 paket berhasil diterima)
=====
```


3.1.2 Analisis Hasil QoS

Berdasarkan data hasil pengujian yang tertera pada Tabel 3.1.1, dilakukan analisis mendalam mengenai performa Quality of Service (QoS) pada jaringan LAN menggunakan protokol UDP dengan membandingkan skenario pembebanan 10 paket dan 50 paket.

Analisis Round Trip Time (RTT): Pada skenario LAN 10 paket, nilai RTT minimum tercatat sebesar 4.75 ms dengan rata-rata (Avg) 68.25 ms. Ketika beban ditingkatkan menjadi 50 paket, nilai RTT minimum relatif stabil di angka 5.73 ms, namun RTT rata-rata justru mengalami penurunan menjadi 44.94 ms. Meskipun secara rata-rata menurun, RTT maksimum (Max) pada skenario 50 paket melonjak drastis hingga 422.97 ms (hampir dua kali lipat dari skenario 10 paket yang hanya 245.14 ms).

Secara teori jaringan, lonjakan RTT Max ini terjadi akibat adanya efek antrean (queuing delay) pada buffer socket sistem operasi. Ketika client membombardir echo server dengan 50 paket UDP secara beruntun, thread pada server mengalami beban komputasi yang lebih padat untuk memproses request dan membalikkannya kembali (echoing). Paket-paket terakhir dalam antrean harus menunggu paket terdahulu selesai diproses, sehingga waktu total perjalanan bolak-balik (RTT) untuk paket tersebut membengkak hingga mencapai titik maksimumnya di 422.97 ms.

- Analisis Jitter: Nilai jitter (variasi delay antar paket) menunjukkan tren yang menarik, di mana skenario 10 paket menghasilkan jitter sebesar 107.72 ms, sedangkan skenario 50 paket menghasilkan jitter yang lebih rendah, yaitu 79.62 ms.

Secara ilmiah, jitter yang tinggi pada jumlah paket sedikit (10 paket) disebabkan oleh fenomena cold start pada socket Python dan inisialisasi jaringan di awal transmisi, di mana paket pertama biasanya membutuhkan waktu pencarian rute atau alokasi memori yang jauh lebih lama dibanding paket berikutnya, sehingga deviasi antar rentang waktu menjadi sangat lebar. Sebaliknya, pada skenario 50 paket, setelah beberapa paket awal berhasil lewat, jalur data pada memori buffer dan socket stream telah berada dalam kondisi "hangat" (warm/sustained state). Fluktuasi waktu antar paket berikutnya menjadi lebih konsisten dan rapat, yang secara matematis menurunkan nilai rata-rata jitter keseluruhan.

- Analisis Packet Loss: Kedua skenario menunjukkan angka Packet Loss sebesar 0.0% (tidak ada paket yang hilang). Walaupun protokol UDP secara teori bersifat unreliable dan connectionless artinya UDP tidak memiliki mekanisme error recovery, jabat tangan (handshake), maupun pengiriman ulang (retransmission) jika paket hilang fakta bahwa packet loss bernilai nol membuktikan bahwa media transmisi fisik berupa kabel LAN yang digunakan kelompok kami memiliki

bandwidth yang memadai dan tingkat isolasi yang sangat baik terhadap gangguan elektromagnetik luar. Stabilitas media kabel memastikan seluruh datagram UDP dapat dihantarkan ke tujuan dengan utuh tanpa terbuang di tengah jalan.

- Analisis Throughput: Nilai throughput mengalami peningkatan dari 2.86 kbps pada skenario 10 paket menjadi 4.53 kbps pada skenario 50 paket. Peningkatan ini sangat selaras dengan teori fungsionalitas jaringan. Protokol UDP memiliki overhead header yang sangat kecil (hanya 8 bytes per paket) jika dibandingkan dengan TCP (20 bytes). Ketika jumlah paket yang dikirimkan meningkat dalam satu satuan waktu yang padat, utilitas jalur data menjadi lebih optimal. Karena tidak ada jeda waktu untuk menunggu konfirmasi (Acknowledgment atau ACK) seperti pada TCP, client dapat memompa data secara terus-menerus. Semakin banyak paket yang dikirimkan secara beruntun, maka volume data efektif yang berhasil ditransmisikan per detik (throughput) akan merangkak naik hingga menyentuh batas saturasi maksimal dari kapasitas skrip socket Python tersebut.

3.2 Analisis Cache HIT vs MISS

Perbandingan latency antara request yang dilayani dari cache proxy (HIT) dengan request yang harus diteruskan ke web server (MISS).

Kondisi	Latency Rata-rata (ms)	Keterangan
Cache MISS (request pertama)	15.26 ms	Diteruskan ke web server
Cache HIT (request berikutnya)	1.85 ms	Dilayani langsung oleh proxy

Berdasarkan hasil tersebut, terdapat selisih latency sebesar:

$$15,26 \text{ ms} - 1,85 \text{ ms} = 13,41 \text{ ms}$$

Artinya, request yang dilayani melalui cache proxy lebih cepat sekitar **13,41 ms** dibandingkan request yang harus diteruskan ke web server. Jika dihitung dari persentase penurunan latency, Cache HIT mampu menurunkan waktu respons sekitar **87,88%** dibandingkan Cache MISS. Dengan kata lain, request Cache HIT berjalan sekitar **8,25 kali lebih cepat** dibandingkan Cache MISS.

Perbedaan latency tersebut terjadi karena pada kondisi **Cache MISS**, proxy harus melakukan beberapa proses tambahan. Proxy menerima request dari client, membuka koneksi TCP baru ke web server, meneruskan request, menunggu respons dari web server, menerima data respons, menyimpan respons ke file cache, lalu mengirimkan respons tersebut kembali ke client. Proses ini membutuhkan waktu lebih lama karena

melibatkan komunikasi jaringan antara proxy dan web server serta proses penulisan cache ke penyimpanan lokal.

Sementara itu, pada kondisi **Cache HIT**, proxy tidak perlu lagi membuka koneksi ke web server. Proxy cukup memeriksa keberadaan file cache, membaca respons yang sudah tersimpan di penyimpanan lokal, lalu mengirimkannya langsung ke client. Karena proses round-trip ke web server dapat dihindari, latency menjadi jauh lebih kecil. Meskipun masih terdapat proses I/O berkas untuk membaca file cache, waktunya tetap lebih rendah dibandingkan proses komunikasi jaringan dan pengambilan respons dari web server.

Dengan demikian, mekanisme caching pada proxy terbukti dapat mempercepat waktu respons request berikutnya dan mengurangi beban web server. Cache HIT memberikan performa yang lebih baik karena resource yang diminta sudah tersedia di sisi proxy, sehingga client tidak perlu menunggu proxy mengambil ulang data dari web server.

3.2 Skenario Single Client vs Multi-Client

3.2.1 Bukti Log Aktivitas Terminal

Berikut adalah rekaman aktivitas log pada *Proxy Server* yang menunjukkan perbedaan penanganan antara permintaan pertama (*Single Client/Initial Request*) dengan permintaan yang masuk secara simultan dari banyak *client* (*Multi-Client*):

- Log Saat Mengakses Path Baru (Single Client - Cache MISS):

```
[LOG] IP: 10.169.91.144 | URL: /index.html | Cache: MISS | Waktu: 1401.30 ms
```

- Log Saat Mengakses Path Bersamaan (Multi-Client - Cache HIT):

```
[LOG] IP: 10.169.91.144 | URL: /index.html | Cache: MISS | Waktu: 66.76 ms
[LOG] IP: 10.169.91.144 | URL: /index.html | Cache: HIT | Waktu: 2.07 ms
[LOG] IP: 10.169.91.144 | URL: /index.html | Cache: HIT | Waktu: 0.51 ms
[LOG] IP: 10.169.91.144 | URL: /index.html | Cache: HIT | Waktu: 0.37 ms
[LOG] IP: 10.169.91.144 | URL: /index.html | Cache: HIT | Waktu: 0.38 ms
```

3.2.2 Analisis Perilaku dan Waktu Respons

Berdasarkan data log terminal di atas, terdapat perbedaan perilaku sistem dan waktu respons yang sangat signifikan antara kondisi *Single Client* (permintaan awal) dan kondisi *Multi-Client* (permintaan konkuren):

- **Karakteristik Perilaku Single Client (Initial Request):** Ketika sistem pertama kali berjalan dan menerima sebuah *request* tunggal dari *client* untuk berkas `/index.html`, waktu respons yang dibutuhkan sangat tinggi, yaitu mencapai **1401.30 ms** (atau **66.76 ms** pada pengujian subsisten). Perilaku ini terjadi karena

status *cache* pada proxy masih kosong (*Cache MISS*). Akibatnya, *thread* penanganan *client* tersebut harus melakukan operasi I/O jaringan jarak jauh (*remote network I/O*), meliputi proses pembuatan *socket* baru, inisialisasi jabat tangan TCP (*3-way handshake*) ke arah Web Server, pengiriman ulang data *request*, hingga penulisan biner respons ke dalam penyimpanan lokal. Latensi jaringan fisik luar dan proses *disk I/O* inilah yang menyebabkan waktu respons *Single Client* di awal cenderung lambat.

- **Karakteristik Perilaku Multi-Client (Akses Bersamaan):** Perilaku sistem berubah drastis ketika skenario berpindah ke *Multi-Client*, di mana banyak utas *client* melakukan request ke *path* yang sama secara simultan. Setelah *client* pertama berhasil memicu penulisan berkas *cache*, status request untuk *client-client* berikutnya langsung berubah menjadi **Cache HIT**. Waktu respons terpendek secara radikal menjadi **2.07 ms**, **0.51 ms**, hingga menyentuh titik paling optimal di **0.37 ms**. Hal ini membuktikan bahwa operasional sistem tidak lagi bergantung pada performa Web Server luar, melainkan langsung dilayani dari memori/berkas lokal proxy.

3.2.3 Dampak Arsitektur Thread-per-Connection

Keberhasilan pemangkasan waktu respons pada kondisi *Multi-Client* ini merupakan dampak langsung dari implementasi arsitektur **thread-per-connection**:

1. **Eliminasi Head-of-Line Blocking:** Jika proxy didesain menggunakan model *single-thread*, proses *Cache MISS* pertama yang memakan waktu hingga 1401.30 ms akan menyandera seluruh antrian *client* lainnya. Namun, berkat pemisahan *thread* baru (`client_thread.start()`) untuk setiap koneksi yang masuk, *client* kedua dan seterusnya tidak perlu menunggu proses penulisan *cache client* pertama selesai. Mereka dapat masuk ke fungsi penanganan secara paralel.
2. **Kecepatan Layanan Konkuren:** Begitu berkas *cache* tersedia di lokal, *thread-thread* baru yang menangani *client* konkuren dapat langsung membaca berkas tersebut secara bersamaan. Alhasil, efisiensi waktu respons di bawah 1 ms dapat dinikmati oleh banyak *client* sekaligus tanpa saling menginterupsi proses *read/write* satu sama lain..

3.4 Evaluasi Multithreading

3.4.1 Bukti Perilaku Thread

Untuk membuktikan bahwa sistem *Proxy Server* berhasil menjalankan penanganan paralel secara *interleaved* (berbarengan), berikut adalah lampiran log aktivitas pada sisi Web Server:

```
[TCP LOG] IP: 10.169.91.44 | File: /index.html | Waktu: 2026-06-14 19:08:01
| Status: 200 OK
[TCP LOG] IP: 10.169.91.44 | File: /index.html | Waktu: 2026-06-14 19:19:56
| Status: 200 OK
```

Analisis Perilaku Thread: Berdasarkan log di atas, *Web Server* merekam jejak masuknya koneksi TCP dari host 10.169.91.44. Melalui arsitektur *thread-per-connection* yang diterapkan pada `main()` menggunakan `threading.Thread(target=handle_client)`, setiap kali ada permintaan koneksi dari *client*, *socket* utama tidak mengalami pemblokiran (*blocking*). Sistem langsung melemparkan fungsi penanganan ke *daemon thread* baru, sehingga *socket* utama dapat segera kembali melakukan `accept()` untuk menyambut request berikutnya. Log waktu di atas membuktikan bahwa server mampu mempertahankan koneksi terbuka dan melayani siklus request-response secara mandiri di dalam utasnya masing-masing tanpa harus mengantre secara sekuensial.

3.4.2 Konsistensi Cache pada Akses Konkuren

Mekanisme *caching* yang diuji secara konkuren menunjukkan perbedaan waktu respons yang sangat masif antara permintaan pertama (*Cache MISS*) dengan permintaan-permintaan simultan berikutnya (*Cache HIT*). Bukti performa ini terekam jelas pada log *Proxy Server* berikut:

```
[LOG] IP: 10.169.91.144 | URL: /index.html | Cache: MISS | Waktu: 66.76 ms
[LOG] IP: 10.169.91.144 | URL: /index.html | Cache: HIT | Waktu: 2.07 ms
[LOG] IP: 10.169.91.144 | URL: /index.html | Cache: HIT | Waktu: 0.51 ms
[LOG] IP: 10.169.91.144 | URL: /index.html | Cache: HIT | Waktu: 0.37 ms
[LOG] IP: 10.169.91.144 | URL: /index.html | Cache: HIT | Waktu: 0.38 ms
```

Analisis Konsistensi & Latensi:

- **Permintaan Awal (Cache MISS):** Memerlukan waktu **66.76 ms** (dan pada pengujian beban puncak tertentu di log lain bahkan dapat menyentuh **1401.30 ms** akibat *overhead* jabat tangan TCP ke server asal). Latensi tinggi ini terjadi karena proxy harus mengalokasikan waktu untuk membuka *socket* baru, melakukan *forwarding* data ke Web Server, menunggu respons biner dari server, dan menuliskan hasilnya ke media penyimpanan lokal.
- **Permintaan Simultan (Cache HIT):** Setelah berkas ter-caching, rentetan *request* berikutnya diselesaikan dalam waktu yang sangat singkat, yaitu **2.07 ms**, **0.51 ms**, hingga mencapai titik optimal di **0.37 ms**. Hal ini terjadi karena data langsung dibaca dari media penyimpanan lokal proxy tanpa melakukan komunikasi jaringan luar (*network I/O*) ke Web Server.

Mekanisme Lock & Pencegahan Race Condition: Ketika banyak *client* mengakses *path* yang sama secara bersamaan, potensi terjadinya *race condition* sangat tinggi—di mana dua *thread* atau lebih bisa mencoba menulis berkas `cache_index.html` secara serentak sehingga menyebabkan berkas korup. Untuk

menjamin konsistensi data, komponen `proxy.py` menggunakan objek Mutual Exclusion (`cache_lock = threading.Lock()`). Melalui sintaks `with cache_lock:`, *thread* yang mengalami *Cache MISS* akan mengunci gerbang penulisan berkas. *Thread* konkuren lain yang berjalan paralel dipaksa mengantre sebelum diizinkan memeriksa keberadaan fisik berkas lewat `os.path.exists()`. Mekanisme pertahanan berlapis ini memastikan integritas data *header* (seperti Content-Type) tetap utuh dan terhindar dari fenomena *corrupted write*.

3.4.3 Skalabilitas dan Resource Usage

Meskipun model *thread-per-connection* terbukti sukses memangkas latensi secara drastis melalui pemanfaatan *Cache HIT* di bawah 1 ms, model ini memiliki keterbatasan dalam hal skalabilitas:

1. **Overhead Pembuatan Thread:** Setiap kali koneksi baru masuk, sistem operasi harus mengalokasikan ruang memori tersendiri untuk *stack* baru dan melakukan *context switching* pada CPU. Jika terdapat ribuan *client* yang terhubung bersamaan, konsumsi memori RAM akan membengkak secara linear.
2. **Batas Saturasi:** Pada beban kerja skala industri, model ini rentan mengalami kelangkaan sumber daya (*resource exhaustion*).
3. **Rekomendasi Skala Besar:** Untuk menangani beban kerja yang jauh lebih besar, disarankan untuk memodifikasi arsitektur konkurensi dengan menerapkan **Thread Pool** (membatasi jumlah maksimal *thread* aktif sejak awal) atau bermigrasi ke model **Asynchronous I/O** (seperti library `asyncio` atau pola *Non-blocking select/epoll*) yang mampu menangani ribuan koneksi persisten hanya dengan satu *thread* tunggal.

3.4.4 Analisis WiFi vs LAN

Sebagai landasan komparasi teoretis, pemilihan medium fisik **LAN (kabel)** dalam pengujian kelompok kami terbukti memberikan jaminan performa yang jauh lebih superior dibandingkan medium nirkabel (**WiFi**):

- **Faktor Fisik Jaringan:** Media transmisi nirkabel (WiFi) beroperasi menggunakan gelombang radio terbuka yang sangat rentan terhadap *interferensi* sinyal frekuensi lain serta pelemahan dinding bangunan (*fading*). Selain itu, WiFi menggunakan mekanisme *contention* (perebutan jalur udara) berupa CSMA/CA, yang memaksa paket data mengantre jika medium udara sedang sibuk.
- **Dampaknya pada QoS:** Faktor eksternal WiFi tersebut akan memicu fluktuasi *delay* yang lebar (Jitter tinggi) serta risiko hilangnya paket data (*Packet Loss*) di udara. Sebaliknya, dengan menggunakan media kabel LAN yang terisolasi secara elektromagnetik, kelompok kami berhasil mengeliminasi seluruh faktor gangguan

luar tersebut. Hasilnya, performa sistem berada pada tingkat efisiensi tertingginya, yang dibuktikan secara riil oleh angka *Packet Loss* yang menyentuh **0.0%** stabil serta waktu pembacaan *Cache HIT* yang sangat konsisten di bawah **1 ms**.

BAB 4 Troubleshooting dan Kendala Implementasi

Tabel berikut mendokumentasikan kendala teknis yang ditemui selama implementasi dan pengujian beserta akar masalah dan solusi yang diterapkan.

Masalah	Penyebab (Akar Masalah)	Solusi & Dampak
CSS dan JS tidak ter-render di browser; hanya HTML polos yang tampil, padahal berkas berhasil diakses (200 OK).	Web server mengirim header Content-Type: text/html untuk SEMUA berkas, termasuk .css. Browser pada standards mode menolak menerapkan stylesheet yang MIME type-nya bukan text/css (strict MIME checking). Gambar tetap tampil karena di-sniff oleh browser.	Menambahkan pemetaan ekstensi → MIME type (MIME_TYPES + get_content_type()) sehingga .css dikirim sebagai text/css dan .js sebagai application/javascript. Dampak: seluruh styling dan skrip ter-render dengan benar.
Berkas tidak ditemukan (404) meskipun berkas ada.	Server dijalankan dari direktori kerja yang salah; path berkas dicari relatif terhadap current working directory.	Menjalankan server dari direktori yang memuat index.html, css/, dan assets/. Dampak: routing berkas statis konsisten.
Tampilan lama tetap muncul setelah perbaikan diterapkan.	Browser menyimpan versi lama pada cache.	Hard refresh (Ctrl+Shift+R) untuk memuat ulang tanpa cache. Dampak: perubahan langsung terlihat.
Jitter bernilai tidak sesuai definisi.	Jitter sempat dihitung sebagai selisih sederhana, bukan variasi delay.	Menghitung jitter sebagai standar deviasi dari selisih RTT antar-paket berurutan, sesuai spesifikasi. Dampak: metrik QoS valid.
Request timeout pada mode UDP.	Paket hilang akibat sifat UDP yang unreliable / latensi tinggi.	Implementasi setTimeout() pada socket client dan penghitungan paket gagal sebagai packet loss. Dampak: pengukuran tetap berjalan tanpa menggantung.

Penggunaan CPU tinggi saat server idle.	Loop utama menggunakan busy-waiting (while True: pass).	Mengganti dengan <code>threading.Event().wait()</code> yang blocking tanpa konsumsi CPU. Dampak: server efisien saat idle.
Berkas default tidak konsisten antara server dan proxy.	Server dan proxy mengembalikan berkas default berbeda.	Menyamakan berkas default ke <code>index.html</code> pada kedua komponen. Dampak: perilaku akses “/” seragam.

BAB 5 Kesimpulan dan Saran

5.1 Kesimpulan

Sistem Client–Proxy–Server berbasis socket Python berhasil diimplementasikan dan diuji dengan baik sesuai spesifikasi rancangan. Web server mampu melayani permintaan berkas statis melalui protokol TCP dengan penanganan MIME type yang akurat, serta menyediakan fungsionalitas UDP echo server sebagai landasan pengujian performa jaringan. Berdasarkan seluruh rangkaian implementasi, pengujian, dan analisis yang telah dipaparkan pada Bab 3, kelompok kami menyimpulkan beberapa capaian utama sebagai berikut:

1. **Efisiensi Mekanisme Caching Proxy:** Implementasi *file-based caching* pada Proxy Server terbukti secara radikal memangkas waktu respons layanan. Permintaan awal (*Cache MISS*) membutuhkan latensi rata-rata sebesar 15.26 ms (bahkan menyentuh 1401.30 ms pada pengujian inisialisasi akibat adanya rentetan proses *remote network I/O*). Sebaliknya, permintaan berulang (*Cache HIT*) berhasil dilayani langsung dari penyimpanan lokal proxy dengan latensi rata-rata hanya 1.85 ms (menyentuh titik optimal hingga 0.37 ms). Mekanisme ini menghasilkan penurunan latensi sebesar 87.88%, yang berarti pemrosesan *Cache HIT* berjalan 8.25 kali lebih cepat sekaligus berhasil mengeliminasi beban komputasi serta lalu lintas jaringan ke arah Web Server.
2. **Karakteristik Parameter QoS UDP:** Pengujian parameter Quality of Service (QoS) pada jaringan kabel LAN menggunakan protokol UDP menghasilkan performa yang sangat stabil. Nilai *Packet Loss* berada pada angka 0.0% untuk skenario 10 paket maupun 50 paket, membuktikan kualitas media fisik kabel yang aman dari interferensi luar. Peningkatan beban dari 10 menjadi 50 paket memicu kenaikan *throughput* dari 2.86 kbps menjadi 4.53 kbps akibat pemanfaatan jalur data yang lebih padat dengan *overhead* UDP yang kecil (8 bytes). Namun, pembebanan ini berbanding lurus dengan pembengkakan *queuing delay* pada buffer *socket*, di mana RTT Maksimum melonjak dari 245.14 ms menjadi 422.97 ms. Sementara itu, nilai *jitter* menurun dari 107.72 ms ke 79.62 ms seiring berjalannya status *socket data stream* yang mulai stabil (*warm state*).
3. **Validitas Arsitektur Multithreading:** Penerapan model konkurensi *thread-per-connection* menggunakan *daemon thread* terbukti efektif dalam menangani kondisi *Multi-Client*. Arsitektur ini sukses mengeliminasi kendala *Head-of-Line Blocking*, sehingga rentetan request simultan dari banyak client dapat ditangani secara paralel (*interleaved*) tanpa saling mengunci atau memblokir satu sama lain. Selain itu, penggunaan `threading.Lock()` (*cache_lock*) berhasil menjamin konsistensi data dan mencegah terjadinya *race condition* atau korupsi berkas cache saat beberapa utas client mencoba menulis file yang sama secara bersamaan.

Keterbatasan Implementasi: Meskipun sistem telah memenuhi seluruh target fungsionalitas, proyek ini masih memiliki keterbatasan operasional. Mekanisme *caching* pada proxy belum dilengkapi dengan kebijakan kadaluarsa otomatis (*Expiry/Time-to-Live*),

sehingga tidak dapat mendeteksi perubahan dinamis pada berkas asal di Web Server secara otomatis sebelum berkas cache lokal dihapus manual. Selain itu, model *thread-per-connection* memiliki batas skalabilitas yang linear terhadap memori RAM dan kinerja CPU akibat adanya *overhead* alokasi *stack thread* baru serta beban *context switching*, membuat sistem ini belum ideal untuk menangani beban kerja masif berskala industri..

5.2 Saran

Berdasarkan keterbatasan sistem yang ditemukan selama proses implementasi dan pengujian, berikut adalah beberapa saran yang dapat diterapkan untuk pengembangan sistem di masa mendatang:

- **Peningkatan Skalabilitas Konkurensi:** Mengganti model *thread-per-connection* dengan arsitektur penanganan koneksi yang lebih hemat sumber daya pada beban tinggi, seperti mengimplementasikan pola **Thread Pool** (pembatasan jumlah utas aktif sejak awal) atau bermigrasi ke model **Asynchronous I/O** menggunakan pustaka *asyncio* atau mekanisme *non-blocking select/epoll*.
- **Penerapan Kebijakan Invalidasi Cache (Cache Lifecycle):** Menambahkan fitur manajemen *cache* yang lebih dinamis pada Proxy Server, seperti mekanisme **Time-to-Live (TTL)** atau validasi berbasis waktu (*expiration date*). Hal ini penting agar proxy dapat mendeteksi perubahan data pada Web Server asal secara otomatis tanpa perlu menghapus berkas cache lokal secara manual.
- **Optimasi Sinkronisasi Akses Berkas:** Meningkatkan keamanan proses penulisan cache melalui penerapan metode **File Locking** yang bersifat atomik di tingkat sistem operasi. Pengembangan ini bertujuan untuk memberikan proteksi yang lebih mutakhir pada kondisi akses multi-client yang sangat padat guna menekan risiko korupsi berkas seminimal mungkin.

DAFTAR PUSTAKA

- Fielding, R., & Reschke, J. (2014). Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing (RFC 7230). Internet Engineering Task Force.
- Postel, J. (1980). User Datagram Protocol (RFC 768). Internet Engineering Task Force.
- Postel, J. (1981). Transmission Control Protocol (RFC 793). Internet Engineering Task Force.
- Kurose, J. F., & Ross, K. W. (2021). Computer Networking: A Top-Down Approach (8th ed.). Pearson.
- Python Software Foundation. (2024). socket — Low-level networking interface. Diakses dari <https://docs.python.org/3/library/socket.html>